Lab4-5 STM32F103RCT6+MPU6050+OLED

1 Hardware connection

The circuit connection part of this case uses stm32f103rct6, mpu6050 (IIC), 0.96 inch OLED (128 * 64 pixels, IIC), eight connection cables, and one USB serial download cable. The software part is written using Keil. FlyMcu is used to burn programs, while SSCOM displays serial ports for printing information.

2 Connection result display

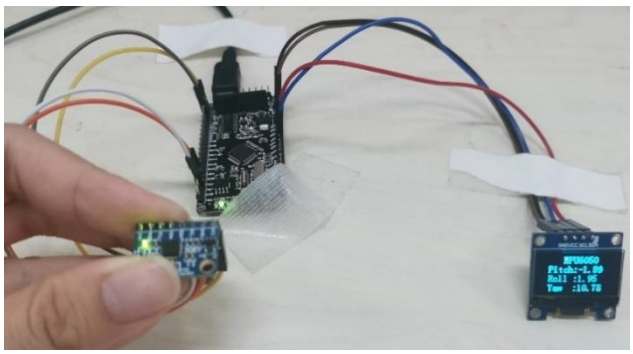

Figure 1 Information after MPU6050 initialization

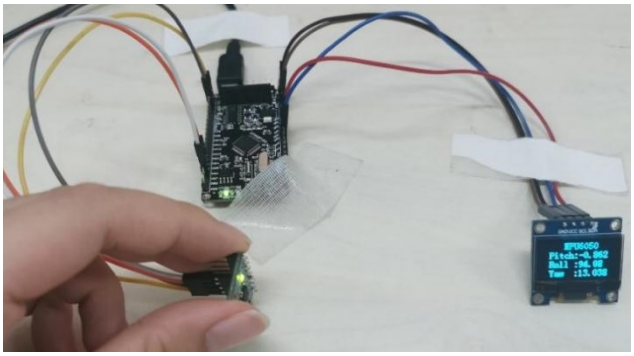

Figure 2 MPU6050 information after rotation

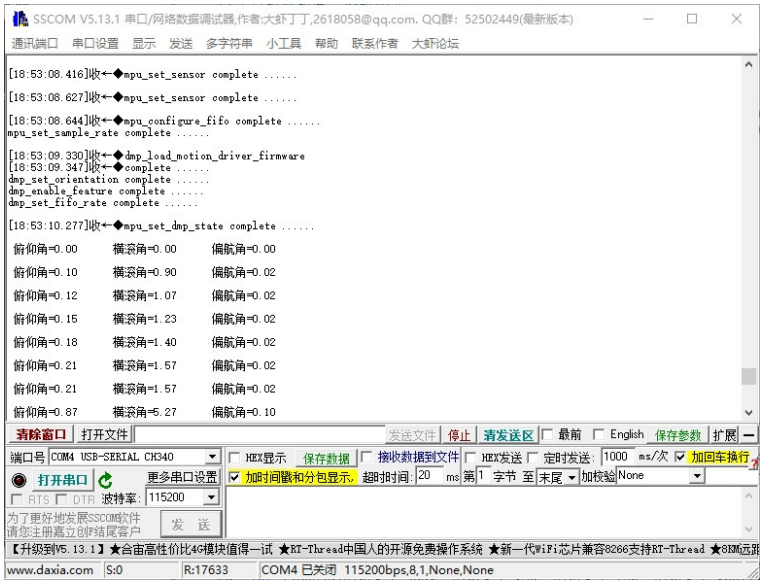

Figure 3 Serial port for printing data

Reference materials

一、OLED 显示屏部分

1. 初始化函数

void OLED_Init(void) —— 使能 OLED 端口时钟，初始化 SCL 与 SDA 所用端口，各项显示设置。

2. 通信函数

void I2C_Start(void) // 起始信号

void I2C_Stop(void) // 结束信号

Figure 8-8 : Definition of the Start and Stop Condition

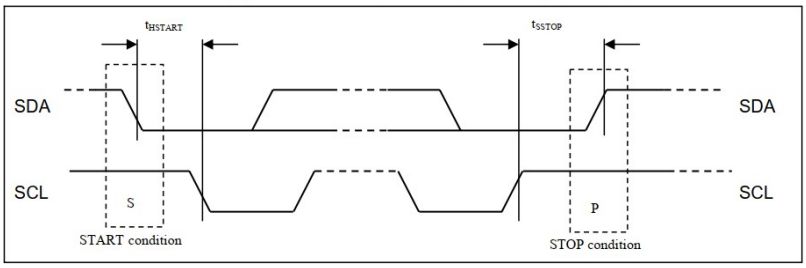

Figure 4 I2C 时序图

void I2C_WaitAck(void) // 等待信号反应，读数据线信号电平判断 Ack 或 nAck

void Send_Byte(u8 dat) // 写入一个字节的数据

void OLED_WR_Byte(u8 dat,u8 mode) // slave address and R/W，display start line(mode), data

3. 命令函数（详情见 SSD1306datasheet.pdf)

void OLED_ColorTurn(u8 i)反显函数（0 正常显示，1 反色显示）

```
//反显函数
void OLED_ColorTurn(u8 i)
{
  if(i==0)
    {
      OLED_WR_Byte(0xA6,OLED_CMD);//正常显示
    }
  if(i==1)
    {
      OLED_WR_Byte(0xA7,OLED_CMD);//反色显示
    }
}
```

Figure 5 反显函数代码

10.1.10 Set Normal/Inverse Display (A6h/A7h)

This command sets the display to be either normal or inverse. In normal display a RAM data of 1 indicates an "ON" pixel while in inverse display a RAM data of 0 indicates an "ON" pixel.

Figure 6 反显设置在手册中的介绍

void OLED_DisplayTurn(u8 i) // 屏幕旋转 180°

void OLED_WR_BP(u8 x,u8 y) // 配置写入数据的起始位置

void OLED_DisPlay_On(void) // 开启 OLED 显示

void OLED_DisPlay_Off(void) // 关闭 OLED 显示

void OLED_Refresh(void) // 更新显存到 OLED

void OLED_Clear(void) // 清屏函数

4. 画图函数

void OLED_DrawPoint(u8 x,u8 y) // 画点（x:0~127，y:0~63）

void OLED_ClearPoint(u8 x,u8 y) // 清除一个点（x:0~127，y:0~63）

void OLED_DrawLine(u8 x1,u8 y1,u8 x2,u8 y2) // 画线（x:0~128，y:0~64）

void OLED_DrawCircle(u8 x,u8 y,u8 r) // 画圈（x，y：圆心坐标，r：圆的半径）

void OLED_ShowChar(u8 x,u8 y,u8 chr,u8 size1) // 显示一个字符（x:0~127，y:0~63，size：12/16/24）
void OLED_ShowString(u8 x,u8 y,u8 *chr,u8 size1) // x, y：起点坐标，size1：字体大小，*char:字符串起始地址
u32 OLED_Pow(u8 m,u8 n) // 计算多次项
void OLED_ShowPicture(u8 x0,u8 y0,u8 x1,u8 y1,u8 BMP[]) // 显示两个数字（x，y：起点坐标，len：数字的位数，size：字体大小）
void OLED_ShowChinese(u8 x,u8 y,u8 num,u8 size1) // 显示汉字（x，y：起点坐标，num：汉字对应的序号）
void OLED_ScrollDisplay(u8 num,u8 space) // 显示多个汉字

二、MPU6050 部分

MPU6050 是六轴运动处理传感器，继承了三轴 MEMS 陀螺仪，三轴 MEMS 加速度计以及一个可扩展的数字运动处理器 DMP，使用它的目的通常是为了得到待测物体的 x、y、z 轴倾角（俯仰角 Pitch、翻滚角 Roll、偏航角 Yaw）。通过 IIC 接口读取到 MPU6050 的六个数据（三轴加速度，三周角速度）经过姿态融合后，我们就可以得到 Pitch、Roll、Yaw 角。

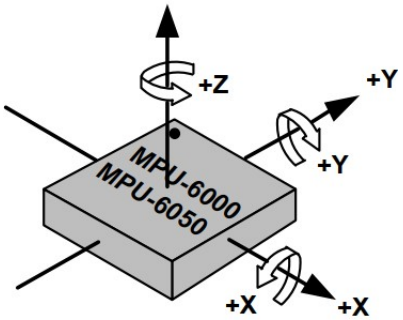

Figure 7 传感器灵敏度轴的方向和旋转极性

陀螺仪传感器在其内部有一个陀螺，因为陀螺效应始终与初始方向平行，我们能通过与初始方向的偏差计算出旋转方向与角度。加速度传感器是一种能够测量加速度的传感器，根据压电效应原理，在加速过程中，测量质量块所受惯性力，再根据牛顿第二定律计算得到加速度值。

数字运动处理器内置卡尔曼滤波算法，可以获取陀螺仪和加速度传感器数据，并处理输出四元数，既减轻外围微处理器的工作负担，又避免了繁琐的滤波和数据融合。

三、数据处理

在从陀螺仪和加速度传感器处得到三轴角加速度和三轴线加速度后，我们需要进行数据的处理和融合，具体步骤如下：

1、校准数据（零点漂移）：传感器安装在设备上总有一个初始的角度，我们设这个角度为 0 度，我们每一次的数据都要减去这个初始数据，得到一个相对的角度。

2、把测量值换算成相应的单位：原始数据除以它在该量程下的灵敏度就可以获得实际的物理单位。加速度的物理单位为 g，角速度的物理单位为°/s。

Cross-Axis Sensitivity vs. Orientation Error

Orientation Error (θ or Φ)	Cross-Axis Sensitivity (sinθ or sinΦ)
0º	0%
0.5º	0.87%
1º	1.75%

Figure 8 MPU6050 手册灵敏度表格

3、滤波和数据融合：常见方法有三种：互补滤波、卡尔曼滤波、硬件 DMP 解算四元

数。
- 互补滤波：因为加速度计有高频噪声，陀螺仪有低频噪声，需要互补滤波融合得到较可靠的角度值。
- 卡尔曼滤波：利用线性系统状态方程，通过系统输入输出观测数据，对系统状态进行最优估计的算法。由于观测数据中包括系统中的噪声和干扰的影响，所以最优估计也可看作是滤波过程。
- 硬件 DMP 解算四元数：DMP 将原始数据直接转换成四元数输出，运用欧拉角转换算法，从而得到 yaw、roll 和 pitch。

任务：

1. 阅读代码, 连接电路, 补充 void Read_DMP(float *Pitch,float *Roll,float *Yaw), 得到 pitch, roll，yaw 角并显示，以串口通信的方式显示结果，拍摄视频留存。
2. 画一张程序框图说明 stm32 如何实现读取角度并显示的。
3. 用 OLED 显示姿态角。
4. 上交实验报告。